

A1

Seat No.



VIT

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

409

Authorization
Seal

Test : CAT-I / CAT-II / Mid-Term

Register No.: 21BCT0179 Course Code: BCSE305L Class NBR: 1452
 Student's Name: Mr/Ms. AVINASH PANDEY Course Title: Embedded Systems
 Programme: B-Tech School: SCOPE Faculty Name: Prof. Ajida D
 Slot: A2+TA2 Session: FN/AN/EVN Date of Exam: 16/03/25 No. of pages written: 7

Signature of the Student

Signature of the Invigilator:

Name / Emp. ID of the Invigilator:

1452

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	3		9			17			25			33		
2	6		10			18			26			34		
3	1		11			19			27			35		
4			12			20			28			36		
5	2		13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total			Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

07

Marks In Words : 210.00

Evaluator's Signature

Date : 23/3/25

Evaluator's Name : Prof. Dr. P. Jeyaraj

(1.) FSM Desired FSM is:-

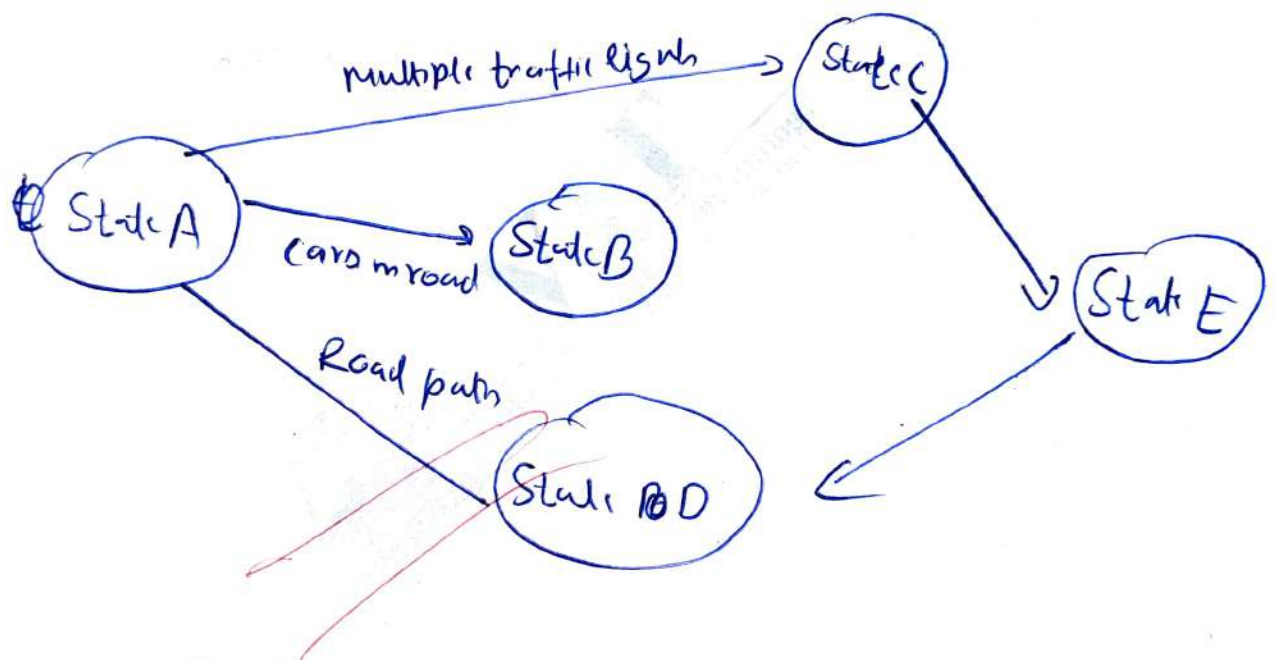
State A:- Autonomous car ready to move from VIT to Chennai

State B:- Encountering cars wrt to the autonomous car

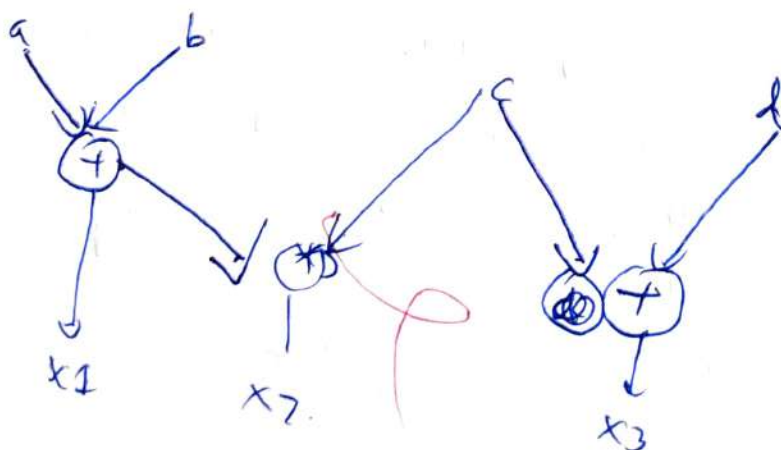
State C:- Multiple traffic light stations

State D:- 9 Road path

State E:- Arrival at the final destination



CD Fw is:-



(3.)
#

Here Task 1 takes 2 sec & is repeated every 4 sec

Task 2 takes 3 sec & is carried out every 8 seconds

Task 3 takes 4 seconds & is repeated every 24 seconds

Assume all tasks arrive at time 0.

Hence schedule based on any 2 models to analyze the constraint for each model with time graph

	arrival	repetition duration	deadline
T ₁	2 sec	4 sec	
T ₂	3 sec	8 sec	
T ₃	4 sec	24 sec	

Here the slack time = $(D - t - e^t)$

at time $t=0$, Task 1 & Task 2 have arrived

at $t=2\text{sec}$

~~T₁ is started and executed by $T_1 = 4\text{sec}$~~

at $t_2 = 3\text{sec}$

T₂ is started

0/

(S)(A) "All embedded systems are not Real time systems."

Ans (S)(A) All embedded systems are not real time systems as embedded systems can be both software based systems as well as hardware based systems (analog system). Now if they can be a program, then they don't have their own physical existence as an entity, so are not real-time systems. eg:- software program to control the

Internal mechanism of a sensor chip has its own inbuilt embedded system which can perform multiple tasks but does not exist for physical touch hence is not a real time sys.

Q.5) B.) identify for Soft RTS or Hard RTS:-

- (1) A Cruise control system:- Soft ~~RTS~~ as it is just a program so no hardware component involved
- (2) An ATM machine:- Hard ~~RTS~~ as it has its own physical hardware presence along with its software
- (3) A Pacemakers Hardware RTS because of the same reason as above
- (4) A Fitbit watch:- Hard RTS
- (5) A Smart fridge:- Hard RTS

(4) For a smart city project
3 processes which happen simultaneously.
So 3 posn process

(2) (a) $m \leq a < b$

(2)(b)

Hardware development
tools

(1) It includes chips, devices
which exist physically.

(2) Not have its own working
system

(3) It is manual

(4) Cannot perform all task
easily

Software development
tools.

It includes program code
designed to perform a specific
task

Has its own system

It is automatic.

It can perform tasks
with ease.

Seat No.

5



VIT

Vellore Institute of Technology
(Deemed to be University under section 3 of UGE Act, 1956)Authorization
Seal

Test : CAT-I / CAT-II / Mid-Term

Register No.: 22BCI0139

Course Code: BCSE3054 Class NBR:

Student's Name: Mr/Ms. Harshvardhan

Course Title: Embedded System

Programme: BTECH CSE School: SCOPE

Faculty Name: Prof. 1457

Slot: A2 Session: EM/AN/EVN

Date of Exam: 16/3/25 No. of pages written:

Signature of the Invigilator: S.T. 10

Name / Emp. ID of the Invigilator: 20906

Signature of the Student

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	4		9			17			25			33		
2	4		10			18			26			34		
3	5		11			19			27			35		
4	4		12			20			28			36		
5	8		13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total			Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks:

25

Evaluator's Signature

Date:

23/3/25

Evaluator's Name: Prof.

Dr. D. Jeyaraj

Marks In Words

Two five

Q5

"All embedded system are not Real Time Systems"

Real Time System - Are those system that work within strict time constraints and provide a worst case time estimate for critical situation

- Correctness of the system does not only depend on logical result of computation but also on the time at which the result are produced

Embedded system - provides a specific function in a much larger system.



But RTS Embedded system has performance deadline and adapts to change in ~~environment~~ surroundings

Ex- Embedded system - Clock (Analog) Tells Time or Phone

RTS Embedded system - Acti A digital smart watch which tracks heart rate temperature and change with time

OR ABS - Anti Lock Breaking system

- X -

A Cruise Control System - Hard RTS

An ATM Machine - Soft RTS [Firm RTS soft/Hard Bath]

A Pacemaker - Hard RTS

A FitBit Watch - Soft RTS

A Smart Bridge - Soft RTS

Soft Real Time System: Missing a Deadline May not Be critical and can be Tolerated to a certain degree

Hard Real Time System - Missing a Program / Task execution Time Deadline can have catastrophic consequences

Now Justification

- A Cruise Control System - Missing a Deadline or Execution Task can be very critical Loss of Life [Hard] RTS

- An ATM Machine - can be Tolerated to some extent [Soft RTS]

A Pacemaker - Loss of Human Life can happen catastrophic consequence [Hard RTS]

A FitBit Watch - Deadline can be missed no such effect Soft [RTS]

A Smart Bridge - Deadline can be missed no such effect Soft [RTS]

- x

Process Application Layer

T-kernel
Process Application

Posix
Process Application

Kernel Application Layer

T-kernel
Kernel Application

Posix
Kernel Application

Service driver

Posix

T kernel
standard
extension

user
created
sub system

T-kernel/os

sm

OS

Features

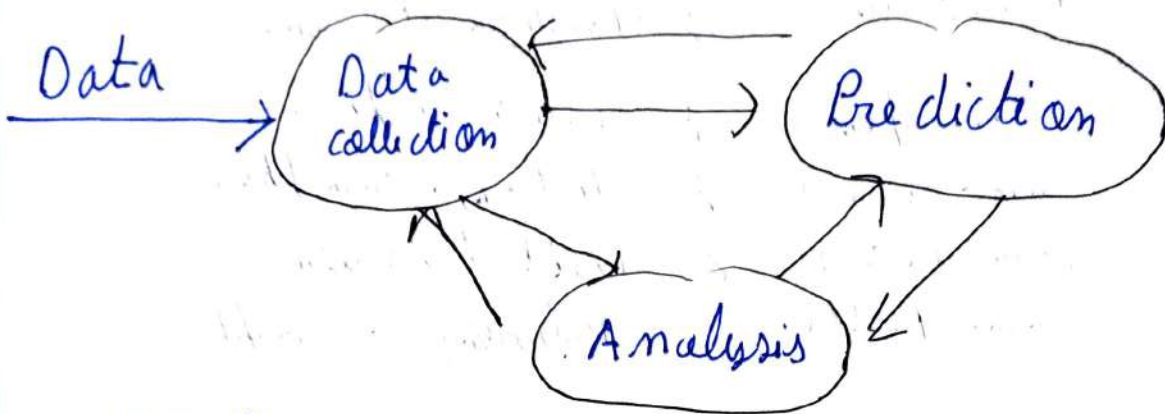
- Multiple Task execute simultaneously
- Thread Synchronisation
- Process need to exchange data
- Inter "communication is Required"
- Use of Message Queue, Shared Memory or signal
- Avoid Race Condition

Standard API

Shared Memory

Allow multiple process to access
of exchange data

Smart City Project



All Process simultaneously.
Poxy reports - Communication will happen B/w all
States

= Shared Memory - Easy Access of Data

Message Queue - Msg waiting until Process Return

Code for cils

FORK() → Process make a copy of It self
child Parent have same code

EXECV() Takes the list of arguments to the Process in form
of Array

Error and exit() Perform call function

wait() make the child process

Posix Semaphores - Support Counting Semaphore

Because Semaphore Allows more than one Process to access a Resource at a Time

sem-open() → Create a Semaphore
sem-close() → Destroy a Semaphore
sem-wait() → Wait for getting
sem-post() → Releasing a Semaphore

Posix Shared Memory - Create Blocks of Memory that can be used by Several Processes

shm-open() → Open a Shared Memory
close → Close a Shared Memory
mmap() → Memory Mapping

Posix Pipes - Return an array of file descriptors

```
if (pipe) (pipe - ends 1 < 0) {  
    error ("pipe")  
    Break;
```

Message Queue → No Need to Create Qm Before. Create a Process

cod

1

1000

1000

1000

1000 1000 1000 1000

1000 1000 1000 1000

1000 1000 1000 1000



VIT
Vellore Institute of Technology
Vellore, Tamil Nadu - 620 019

1000 1000 1000 1000



Q3

Process

Execution Time

Period
4 T_1

2

8 Sec

 T_2

3

 T_3 $\angle \text{CM}$

4

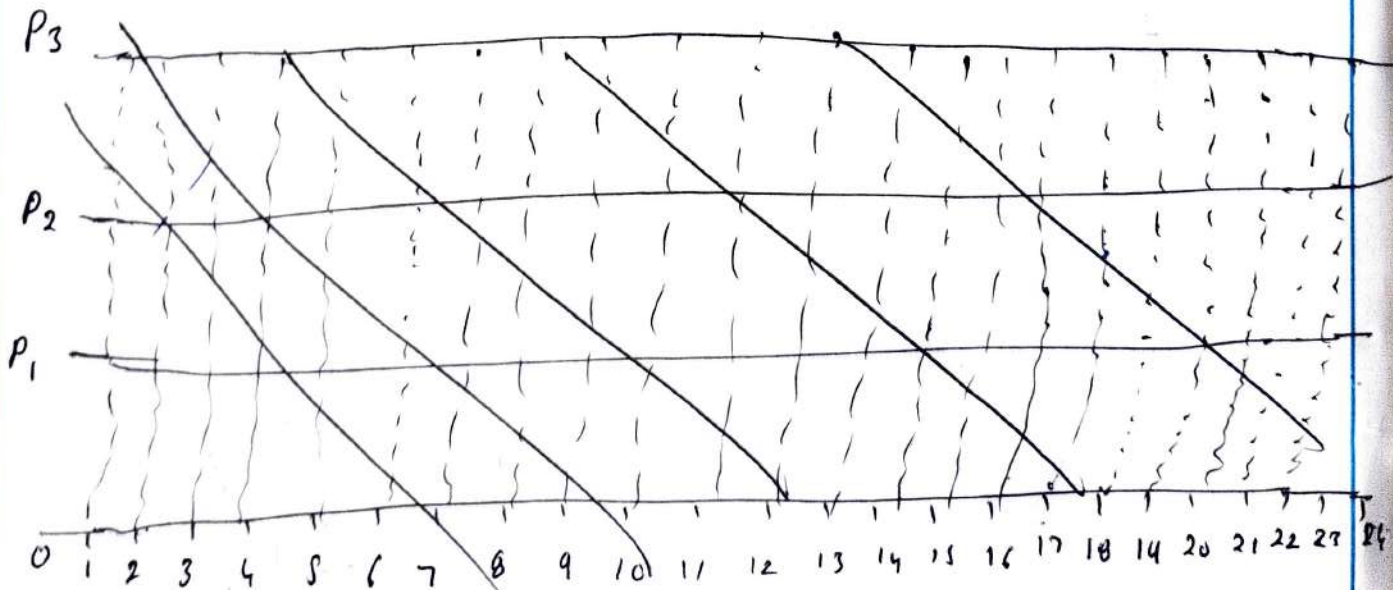
24 Sec

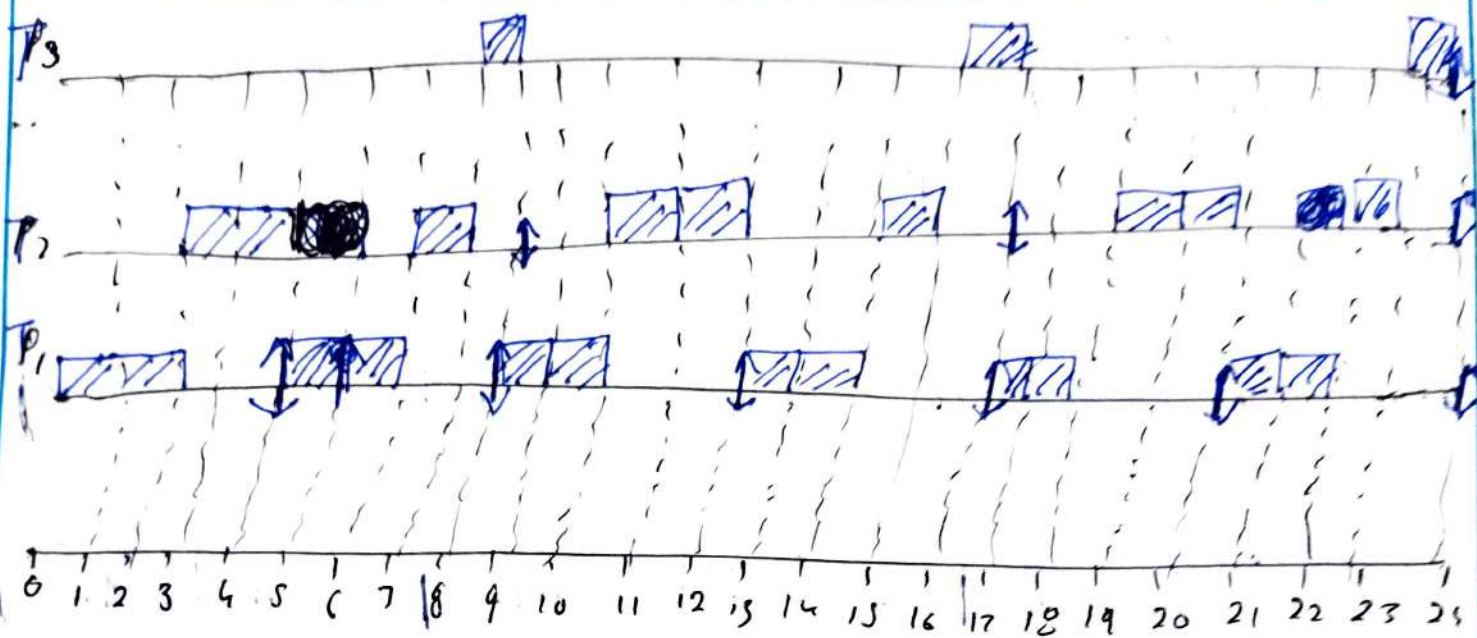
$$\angle \text{CM of Period} = (4, 8, 24) \\ = 24$$

By

Rate Monotonic Scheduling (RMS)

↓ Lowest Priority = Highest Execution Time
 ↑ Highest Priority = Lowest Execution Time





Priority order.

$T_1 > T_2 > T_3$

Based on execution time

Deadline = Period as Not given in question

$$\text{CPU utilization} = 6 \times 2 = 12$$

$$3 \times 3 = 9$$

$$\frac{12}{21} = \frac{24}{24} = 100\%$$

feasibility =

Not feasible

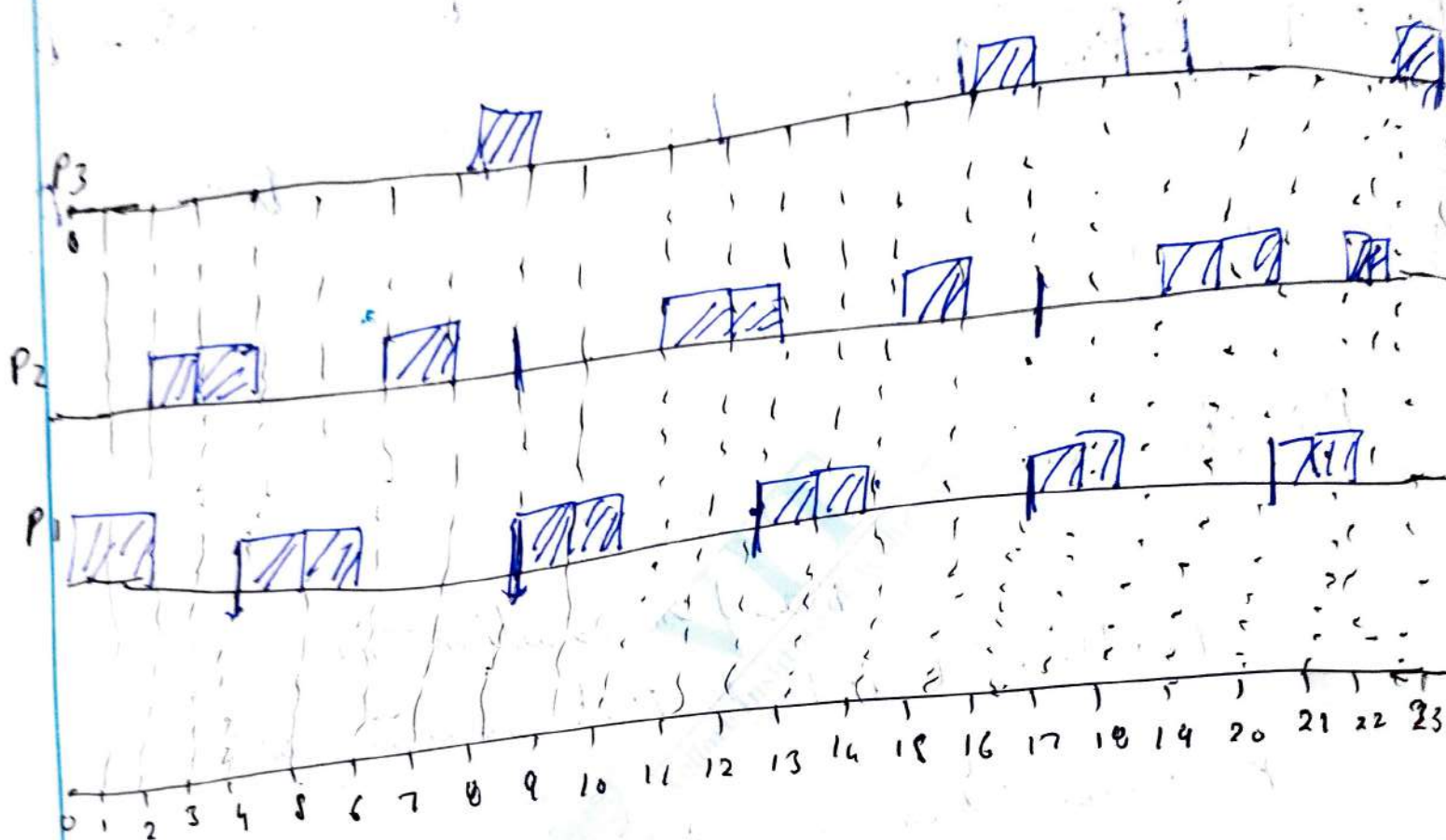
feasibility > 0.7

all

EDF
Round robin = priority

$\angle CM = 24$

P_1, P_2, P_3



Same as max RMS

CPU utilization = 100%

feasibility:

$$2/4 + 3/8 + 4/24$$

$$\frac{12 + 9 + 4}{24} = 24$$

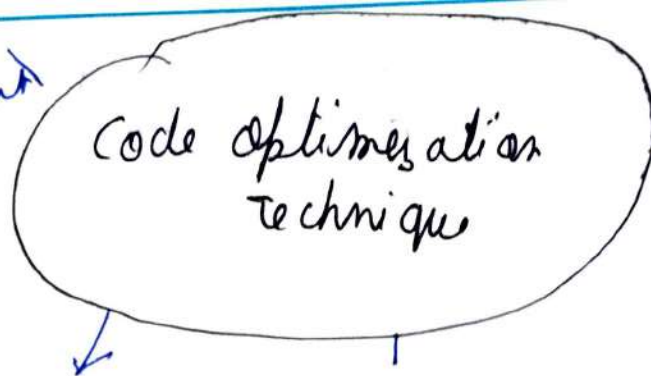
feasibility > 1
Not feasible

Not feasible

So It is not correct was

Q2

Code
manipulation



- Dead Code
Elimination
Common
Sub Expression

Strength
Reduction

Compile
time Evaluation

Constant
Folding

Constant
Propagation

Optimised code

```
# Include <stdio.h>
int main()
```

```
{
  int i;
```

```
  for (i = 2; i <= 100; i++)
```

```
    isPrime = 1;
```

```
    printf("Prime number up to 100:");
```

```
    for (j = 2; j < i; j++) {
```

```
      if (i % j == 0) {
```

```
        isPrime = 0;
```

```
        break;
```

```
      }
```

- Dead Code Elimination

b Hardware development Tools

Tool

Ex

b Hardware Programming Tools: 64 Bit Microcontroller
Ai enabled chip

- Embedded Development
- Emprovid Simulation
- Advance RTOS.

Software development Tools

- Editor -
- Compiler / Assembler
- Linkers
- Debugger
- Simulator

Main Difference is Software development Tools are used for development of Software

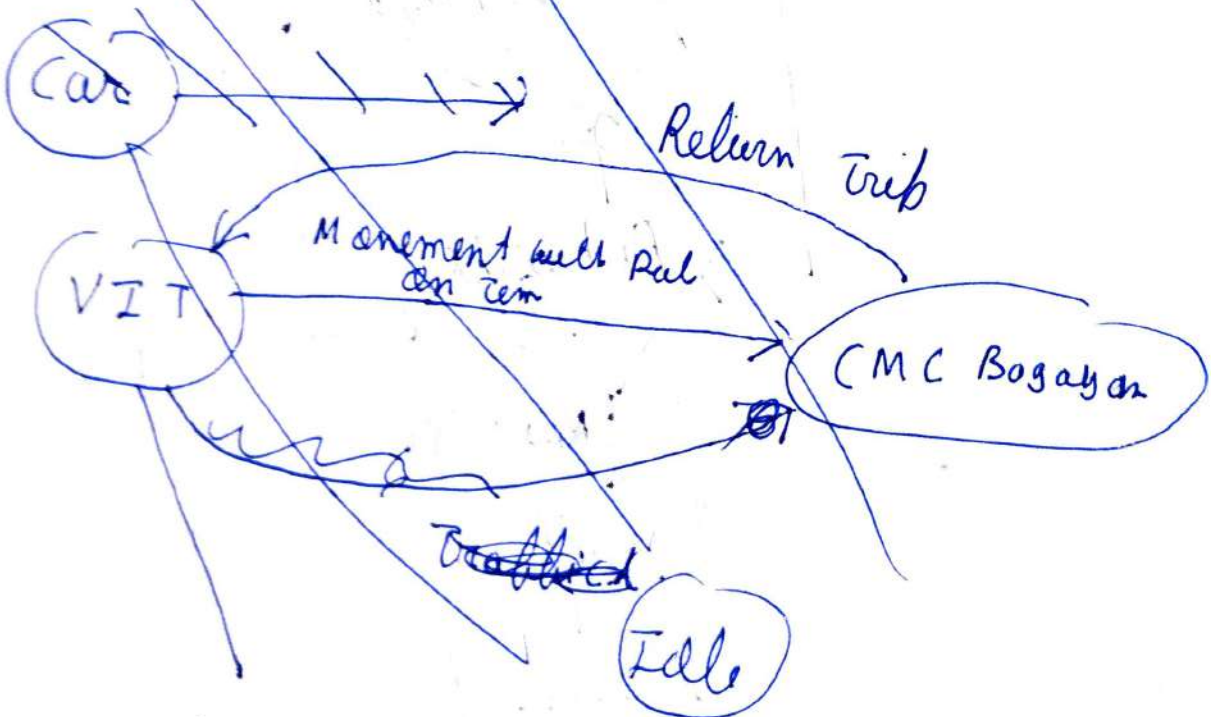
Hardware development is done to improve Software Development Tools
so that Better and Faster Easy with Resuable code.

- A desktop of RTOS.

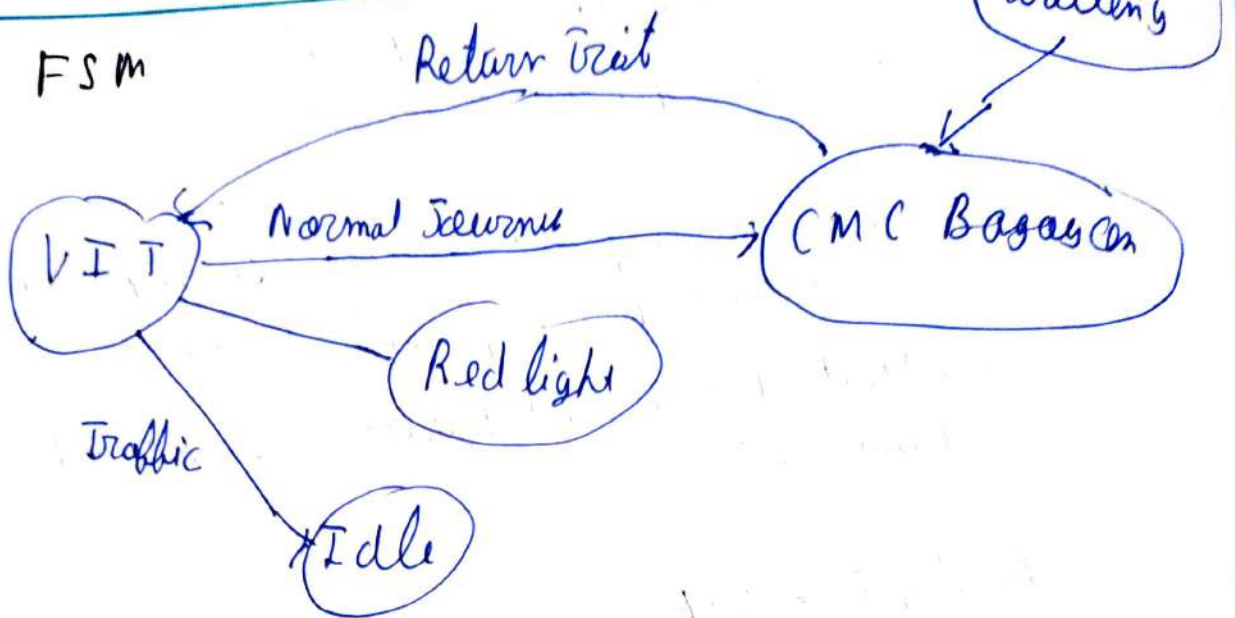
Q1

FSM

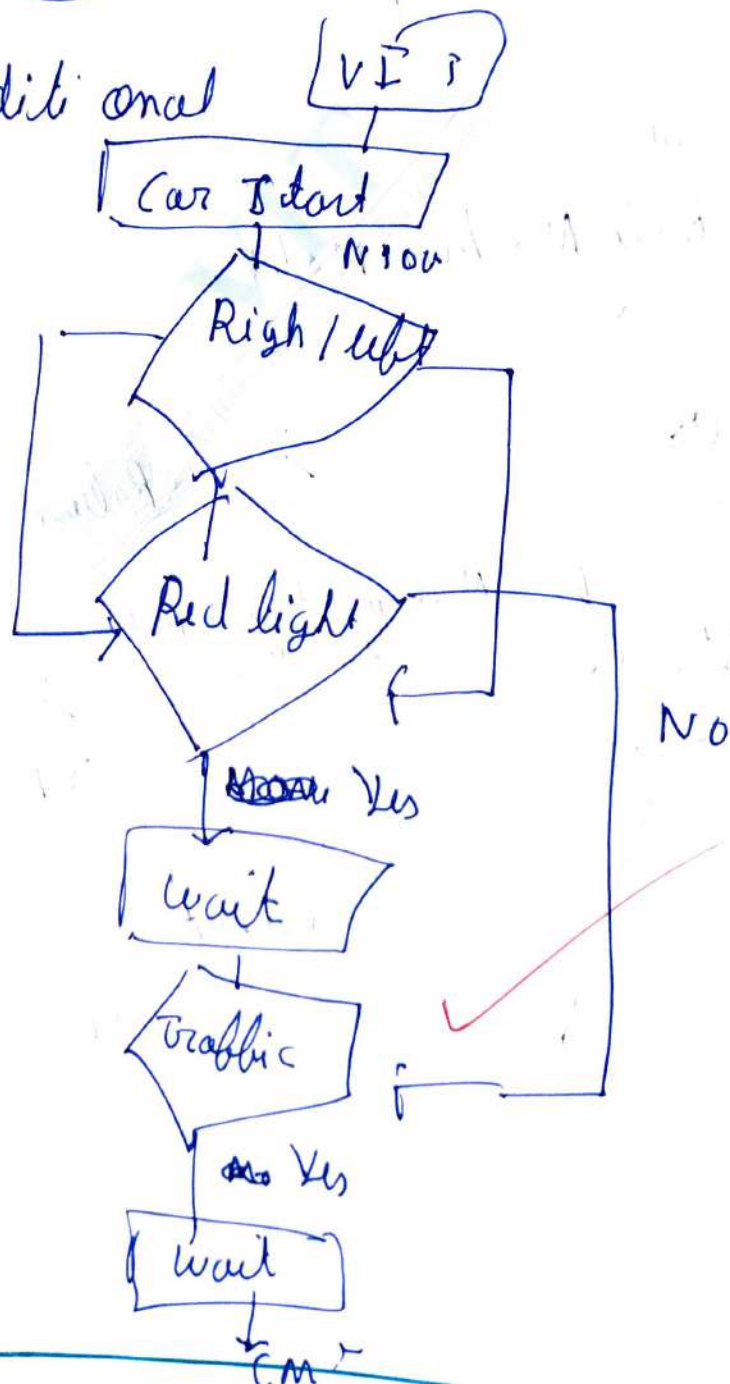
= State Machine Model



Q1 FSM



CDFA Conditional



Seat No.



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Authorization
Seal

Test : CAT- I / CAT- II / Mid-Term

Register No.: 22BCB0003

Course Code: B0E30SL Class NBR: 1457

Student's Name: Mr/Ms. Sruthi Suresh

Course Title: Embedded Systems

Programme: BTech School: SCOPE
CSE w/ Bioinformatics

Faculty Name : Prof. : Ajitha D

Slot : A2 + TA2 Session : FN/AN/EVN

Date of Exam : 16.03.24 No. of pages written : 9

Signature of the Student

Signature of the Invigilator :

Name / Emp. ID of the Invigilator :

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b
1	10		9			17			25		
2	9		10			18			26		
3	10		11			19			27		
4	8		12			20			28		
5	10		13			21			29		
6			14			22			30		
7			15			23			31		
8			16			24			32		
Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

47

Evaluator's Signature

Date :

Evaluator's Name : Prof.

Marks In Words

four seven

Dr. D. Ajitha

2

Code:

#include <stdio.h>

int main()

int i, j, isPrime;

printf("Prime numbers upto 100: "); // only needs to be printed once

for (i = 2; i <= 100; i++)

{ isPrime = 1; // This is a flag - cannot be moved. → reduce no. of iterations

for (j = 2; j <= i/2; j++)

if (i % j == 0) // No need to separately calculate i % j

{ isPrime = 0; break;

}

if (isPrime)

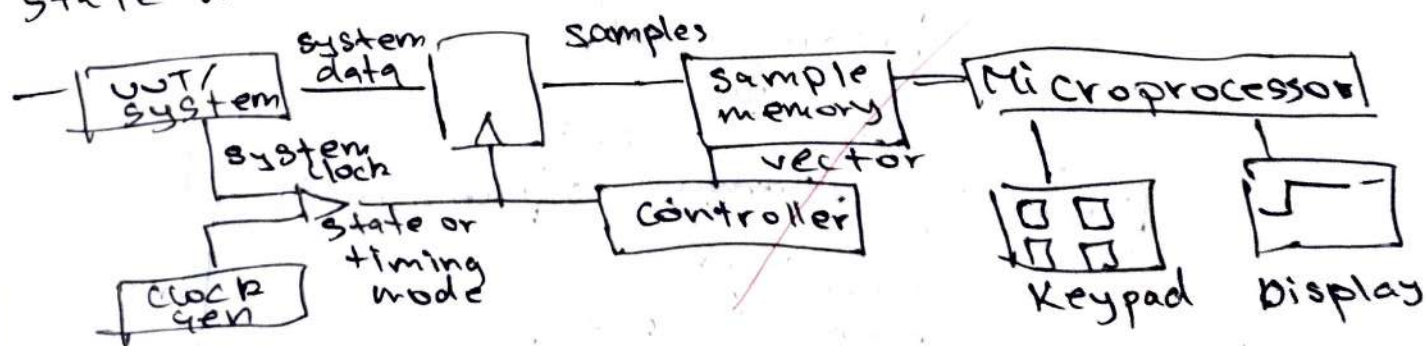
printf("%d\n", i); // loop fusion

}

return 0;

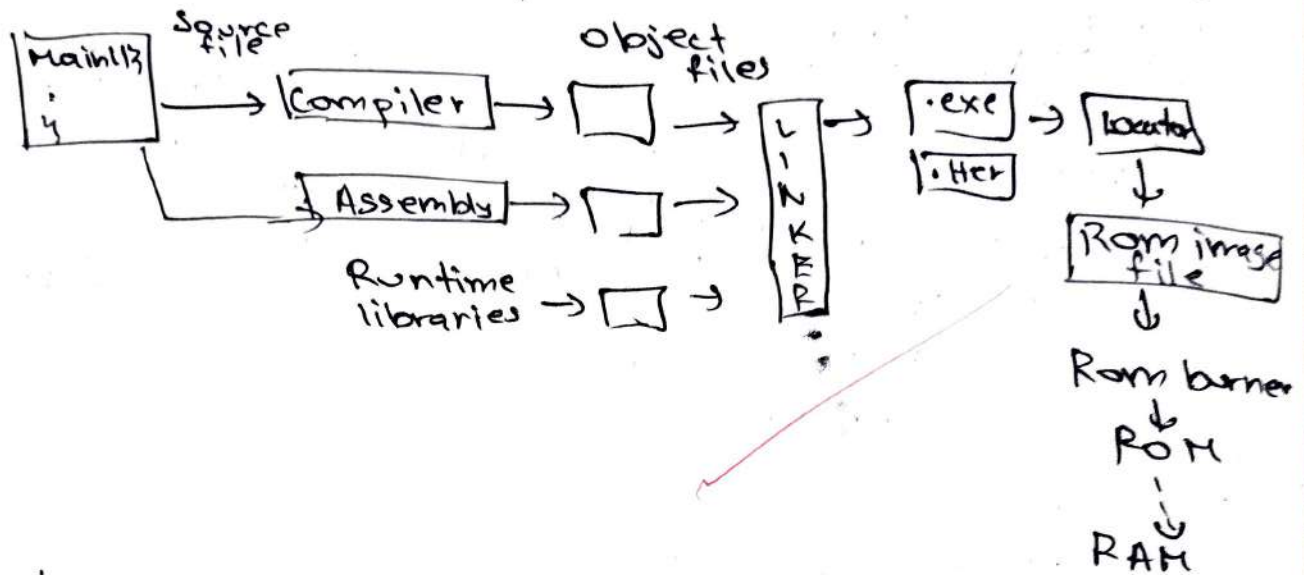
}

Logic analyzer - hardware development tool - It is an electronic measurement device that captures and displays multiple signals from a digital design. It helps with checking and debugging ICs, digital systems, circuits such as embedded systems, electronic control units, computers etc. It converts captured data into timing diagrams, protocol decodes, state machine traces and assembly language.



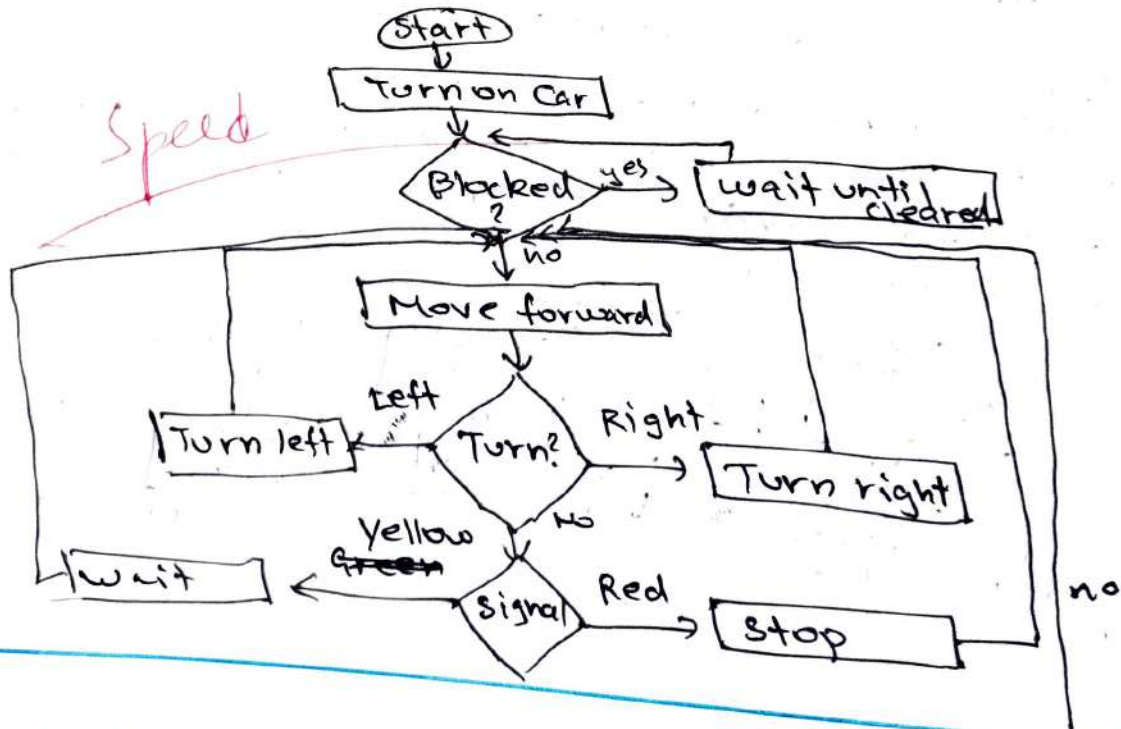
Software development tools:

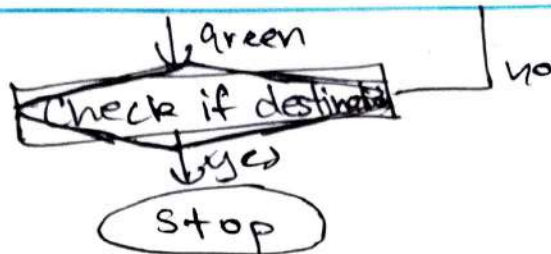
Editor, assembly / compiler, linkers, debuggers, simulators, locators, downloaders and IDEs.



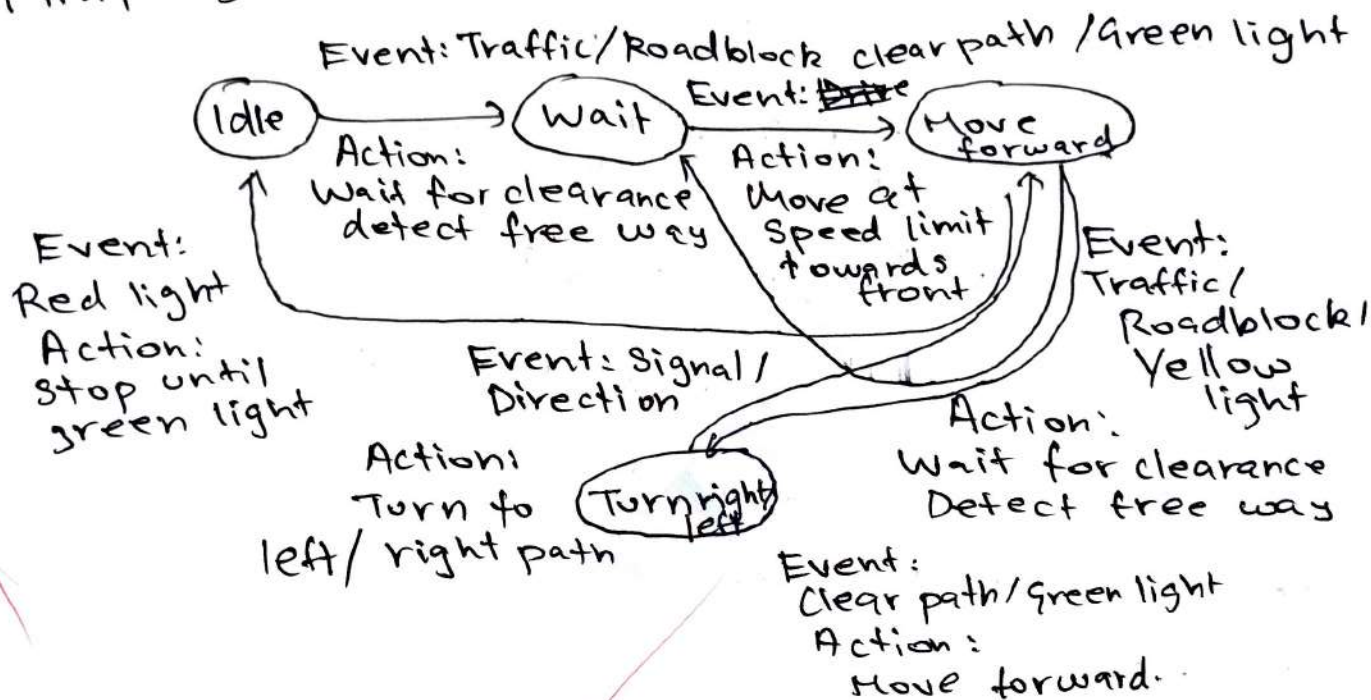
Hardware development tools work with signals voltage and removes noises or distractions. It can only see the visible signals. Software development tools work with source code, which is usually clumsy and implicit, and derives intermediate representations to manipulate and optimize the program.

Control Flow diagram





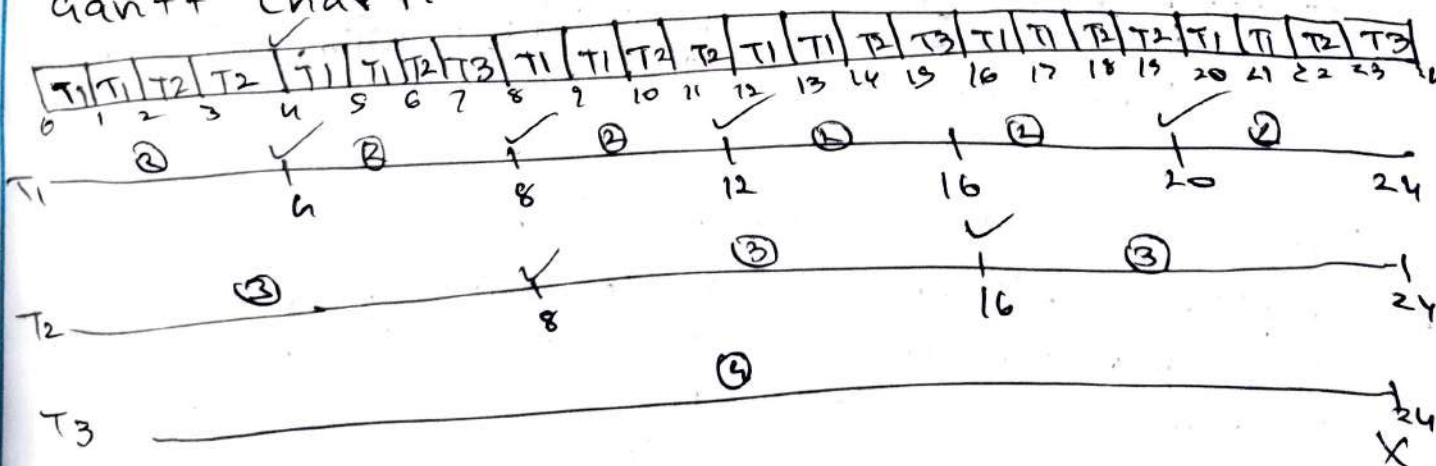
Finite state machine



3) Earliest Deadline First scheduling

	capacity	Deadline	Period	
T ₁	2	4	4	LCM (4, 8, 24) = 24 Hyperperiod = 24
T ₂	3	8	8	
T ₃	4	24	24	

Gantt chart:



Task 3 could not be completed.

Rate Monotonic

$$U = \sum_{i=1}^n \frac{e_i}{p_i} \leq n(2^{1/n} - 1)$$

$$T_1 (4, 2), T_2 (8, 3), T_3 (24, 4)$$

$T_1 > T_2 > T_3 \rightarrow$ fixed priority

$$\sum_{i=1}^n \frac{e_i}{p_i} = \frac{2}{4} + \frac{3}{8} + \frac{4}{24} = \frac{12+9+4}{24} = \frac{25}{24} = 1.041$$

$$n(2^{1/n} - 1) = 3(2^{1/3} - 1)$$

$$= 3(1.2599 - 1)$$

$$= 0.7797$$

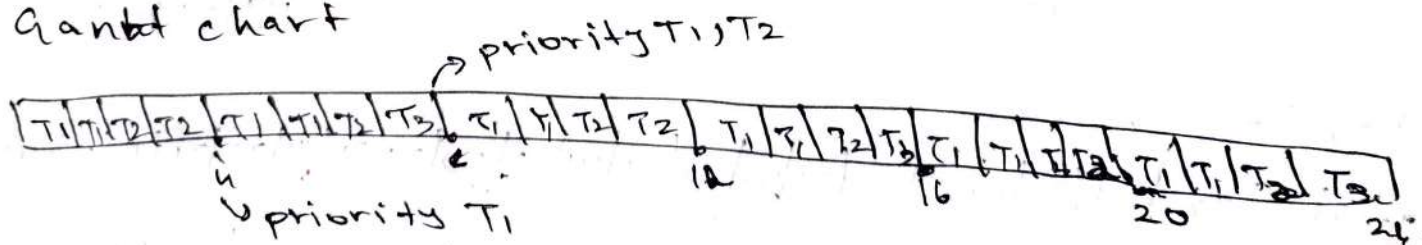
CPU not sufficient.

Utilization

$U = \text{CPU time for useful work} / \text{total available time}$

$> 100\%$ even when excluding scheduling overhead. $\rightarrow$ insufficient $\rightarrow$ cannot be completed

Gantt chart



Priority changes every time slice

Assumptions: All processes run periodically on a single CPU, context switching time is ignored, There are no data dependencies between processes and all deadlines are at the end of their periods, execution time is constant

4) POSIX scheduling can be done either as FIFO, fixed priority scheduling policy, with at least 32 priority levels, or Round Robin with time slices to schedule same priority or implementation defined scheduling policy.

To create three processes simultaneously using huge data, we can use counting semaphores, which allow more than one process to access the resource at the same time $\rightarrow$ if n resources are available, n processes can be allowed before locking.

Shared memory can also be used to access data and update it across multiple processes.

- POSIX-SEMAPHORES

- POSIX-SHAREP_MEMORY-OBJECTS.

objdesc = shm_open("/memobj", O_RDWR)

Example

```
#include <pthread.h>
```

```
#include <stdio.h>
```

```
#include <semaphore.h>
```

```
sem_t mysem;
```

```
void *threadfunction(void *arg) {
```

```
    sem_wait(&mysem);
```

```
    printf("Thread %ld acquired semaphore\n", pthread_self());
```

```
    // critical section
```

```
    // Any data modification
```

```
    sleep(1);
```

```
    printf("Thread %ld releases semaphore\n", pthread_self());
```

```
    sem_post(&mysem);
```

```
    return NULL;
```

```

int main() {
    pthread_t pthread_t t1, t2, t3;
    sem_init(&m_sem, 0, 1);
    pthread_create(&t1, NULL, thread_function, NULL);
    pthread_create(&t2, NULL, thread_function, NULL);
    pthread_create(&t3, NULL, thread_function, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    sem_destroy(&m_sem);
    return 0;
}

```

B) Cruise control system - Hard RTS - failure to meet deadline can cause equipment damage (crash) and potential human injury.

ATM: ~~Firm RTS~~ ^{→ leaning towards Hard.} Failure does not cause system failure, but it makes the usefulness of the output useless. ~~(Hard RTS - sometimes can cause bank~~
 Pacemaker: Hard RTS - failure to meet deadline causes human injury or even death.

Fitbit watch: Soft RTS: user may notice, but no system failure, response still valid.

Smart Fridge: Soft RTS: System is less restrictive, but operations degrade if results are not produced according to specified timing requirement.

A) Not all embedded systems are real time ~~operation~~ systems. A real time system is a system that processes information and responds to events within a specified time constraint. They are in applications where timing is critical. They meet strict deadlines, respond to inputs from

environment and produce outputs. They are highly reliable. For example, in a car, autonomous driving is a real time system, but the air conditioner is not. ABS is a real time system, but the sunroof mechanism is not. This does not mean they do not react to input, they simply do not have a tight time restriction. Many kids toys are not reactive to input at all, and the cheaper ones do not have a predictable, deterministic behavior. There is no necessity for time sensitive response from these. Most standalone embedded systems are not real time systems

